
Same Padding Does Not Preserve the U-Net Geometry and Boundary Contract: A Bounded CPU Study

Abstract

Replacing valid convolutions with zero-padded same convolutions preserves the size of each convolutional feature map, but does it preserve a U-Net’s spatial contract? We separate output extent, skip compatibility, alignment, and boundary response in a four-level encoder–decoder. A scalar study retains the original channel schedule and evaluates eleven square input sizes under two padding modes and three crop policies. Of 66 configurations, 39 are rejected; same padding without cropping succeeds only at size 256 among the tested sizes. A floor-left/remainder-right crop makes all tested sizes executable, while changing their output extent and alignment policy. Separately, 48 CPU forward evaluations use a reduced-channel, fixed-positive-weight network. For an all-ones 256×256 input, valid convolutions return a uniform 68×68 response of 1, whereas same convolutions return 256×256 with a corner response of 0.1162686492 and interior maximum of 1. Structural dependency propagation agrees with all 36 selected impulse checks. These results reject an unrestricted semantics-preserving substitution: size compatibility and boundary behavior require explicit policies. They neither reproduce trained biomedical segmentation nor establish a quality disadvantage of same padding.

1 Introduction

A spatial network exposes more than a tensor shape. Its interface determines which inputs can be processed, how encoder features align with decoder features, which input context is available to each output, and what happens outside the image. Changing a convolution from valid to same padding can simplify one part of that interface while changing another. The question in this working paper is whether that substitution alone preserves a U-Net geometry and boundary contract, including odd and otherwise incompatible input sizes.

The original U-Net combines unpadded convolutions with cropped skip features and an overlap-tile strategy [Ronneberger et al., 2015]. We take its spatial topology as a reference and examine a precisely specified zero-padded alternative. The proposition under test is a potential implementation assumption, not a claim attributed to the original authors or a survey of contemporary U-Net software. In particular, preserving the dimensions of an individual convolution does not imply that four stride-two pooling operations can be inverted by exact doubling at every skip connection.

Our study gives a conditional answer rather than a ranking of architectures. First, dimension recurrences explain why padding and crop policy jointly determine output extent and executability. Second, a complete finite sweep retains rejected configurations as evidence about the contract. Third, a separate fixed-weight numerical network exposes how boundary response and structural support change. These three objects answer different questions: scalar dimensions cannot establish numerical equivalence, and numerical boundary probes cannot establish segmentation accuracy. The central result is that explicit alignment can repair a shape mismatch without restoring the original spatial operator.

2 Spatial contract and reference construction

We define the contract by four components: admissible input dimensions, output dimensions, the left/right crop at each skip connection, and the dependence of outputs on real input pixels versus padded values. The study uses square inputs; the same dimension arithmetic applies separately to the two axes of a rectangular tensor, although rectangular numerical cases were not run. Spatial indices are zero-based. No dilation or input padding is introduced, pooling uses floor division, and each up-convolution doubles spatial extent exactly.

2.1 Dimension recurrence

Let n be an input side length and let $b = 4$ for a block of two valid 3×3 convolutions, or $b = 0$ for two same convolutions. Each same convolution pads one zero on each side. With $t_0 = n$, the encoder block outputs and subsequent pool inputs obey

$$e_\ell = t_\ell - b, \quad t_{\ell+1} = \lfloor e_\ell / 2 \rfloor \quad (\ell = 0, 1, 2, 3), \quad e_4 = t_4 - b. \quad (1)$$

Set $d_4 = e_4$. At decoder level $\ell = 3, 2, 1, 0$, define

$$u_\ell = 2d_{\ell+1}, \quad \Delta_\ell = e_\ell - u_\ell, \quad d_\ell = u_\ell - b. \quad (2)$$

The skip tensor is reduced from e_ℓ to u_ℓ before concatenation. Nonpositive convolution outputs and negative Δ_ℓ are invalid. The final 1×1 projection does not alter d_0 .

We compare three policies. *No crop* requires $\Delta_\ell = 0$ at every level. *Symmetric-only crop* requires every Δ_ℓ to be nonnegative and even, removing $\Delta_\ell/2$ from either side. *Asymmetric crop* removes $\lfloor \Delta_\ell/2 \rfloor$ on the left/top and the remainder on the right/bottom. Thus a one-pixel mismatch has a specific orientation; “center crop” without a rounding rule would not fully specify this experiment.

For same padding, repeated floor pooling gives $t_4 = \lfloor n/16 \rfloor$ and hence, under an admissible crop policy,

$$d_0^{\text{same}} = 16 \lfloor n/16 \rfloor. \quad (3)$$

Exact doubling recovers the discarded pooling residues only when they are all zero. No-crop compatibility therefore requires n to be divisible by 16. A symmetric-only crop has the same restriction in this graph: at the deepest skip the difference is either zero or one; rejecting an odd difference forces zero, and the argument repeats outward. Same convolutions alone do not repair pooling residue.

For valid padding, applying Equation 1 four times gives $t_4 = \lfloor (n - 60)/16 \rfloor$. Equation 2 then yields

$$d_0^{\text{valid}} = 16 \lfloor (n - 60)/16 \rfloor - 124, \quad (4)$$

provided the intermediate dimensions remain positive and the crop policy admits the skips. When every prepool dimension is even, $n \equiv 12 \pmod{16}$ and this reduces to $d_0 = n - 184$. This explains the $572 \rightarrow 388$ anchor. A single subtraction of 184 is not the general formula for arbitrary inputs under floor pooling: at $n = 256$ the output is 68, not 72. For these positive-size cases, symmetric valid cropping imposes the same even-prepool condition; valid convolutions necessarily require nonzero skip cropping.

2.2 What is preserved from the original architecture?

The original architecture description specifies paired unpadded 3×3 convolutions, ReLU, 2×2 stride-two max pooling, channel-reducing up-convolutions, cropped concatenation, and a final 1×1 projection [Ronneberger et al., 2015, Section 2]. Its Figure 1 supplies the $572 \rightarrow 388$ sizes and the 64–1024 channel schedule. The scalar trace here preserves that schedule: encoder level ℓ has $64 \cdot 2^\ell$ channels; the up-convolution halves channels, the matching skip doubles them by concatenation, and the convolution block returns to $64 \cdot 2^\ell$ channels. The final output has two channels. These are scalar counts, not allocated original-width feature maps.

The original even-prepool condition and mirrored context for overlap tiling are essential distinctions [Ronneberger et al., 2015, Section 2 and Figure 2]. Our floor-pooling stress cases extend the admissible-size question beyond that original condition, and our same variant inserts zeros at each convolution. It does not implement the original mirrored input context. Failure on such a stress case is consequently a failure of the tested substitution contract, not a contradiction of the original design.

3 Methods: geometry, signals, and structural support

The empirical evidence consists of a supplied deterministic CPU run with a protocol fixed before execution. The scalar sweep covers $\{188, 189, 190, 200, 252, 253, 254, 255, 256, 257, 572\}$, both padding modes and all three crop policies: 66 configurations in total. All successful layer dimensions and crop offsets are retained, and rejected configurations retain the first shape or crop error. The 572 case is scalar only; no numerical array of that spatial size was allocated.

Numerical network. The forward study uses encoder channels 1, 2, 4, 8, 16 and two identical final output channels. Each 3×3 convolution averages over all incoming channels and the nine spatial entries, then repeats the result across its output channels and applies ReLU. Equivalently, every convolution weight is $1/(9C_{\text{in}})$, with zero bias. The normalized positive construction avoids learned cancellation and makes an all-ones response interpretable; its repeated channels are not a realistic learned feature basis. Pooling takes the maximum in each nonoverlapping 2×2 cell, dropping a trailing row or column if necessary. The up-convolution averages channels and replicates that mean to each output channel and each position of a 2×2 block. This is a fixed 2×2 , stride-two transposed-convolution equivalent with positive unit spatial weights and normalized channel mixing. Decoder skips use asymmetric cropping. The last projection averages channels and repeats the result twice, without a softmax. Reported responses are activations, not class probabilities.

Six input sizes, $\{188, 189, 252, 253, 256, 257\}$, are evaluated in both padding modes. Each receives an all-ones image and three unit impulses on an otherwise zero image: at $(0, 0)$, $(3, 3)$, and $(\lfloor n/2 \rfloor, \lfloor n/2 \rfloor)$. This gives 48 numerical evaluations, including 36 impulse evaluations. Arrays use NumPy’s default float64 for these inputs. The supplied run records Python 3.12.14 and NumPy 2.3.5 on x86_64 Linux, with BLAS/OpenMP thread settings fixed to one. No training, random weight seeds, biomedical images, or GPU execution enter this study.

Structural versus numerical dependence. For each input-axis location i , initialize a bitset containing only i . A convolution takes the union of three neighboring sets; padded zeros contribute the empty set. Pooling unions the two possible contributors on an axis, the up-convolution repeats sets, and a concatenation unions decoder and cropped-skip sets. Applying both axes gives the graph’s possible spatial support. This construction follows potential paths through max pooling, not the particular argmax selected for one image, and it does not estimate gradients or a learned effective receptive field.

Support is recorded at output positions 0, $\lfloor d_0/2 \rfloor$ and $d_0 - 1$ for each numerical configuration, giving 36 structural records. A separate set of 36 checks asks whether the center output’s structural set contains each impulse location and whether the measured center response exceeds the predeclared 10^{-14} threshold. The zero-input output is zero by construction. Agreement in this check validates selected dependency predictions only; it is not an exhaustive pixelwise support test. The stored arrays retain tiny nonzero responses below ordinary display precision.

4 Results

4.1 Same padding preserves neither arbitrary-size compatibility nor extent

Figure 1 shows every scalar configuration, including all 39 rejections. The asymmetric policy admits all eleven sizes in either padding mode. Symmetric valid cropping succeeds at 188, 252 and 572; no-crop valid concatenation succeeds at none of them. For same padding, both no-crop and symmetric-only policies succeed only at 256 in this sweep. These counts describe a deliberately chosen grid, not the prevalence of failures among application images.

The failures expose two different locations of mismatch. With same padding and $n = 252$, the deepest skip has side 31 while its decoder partner has side 30. The mismatch is already present before the first decoder convolution. At $n = 257$, the deepest stages align, but the outermost skip has side 257 against a decoder side of 256. Both differences are odd, so strictly symmetric cropping also rejects them. The even input 252 shows that checking only input parity is insufficient; all four pooling stages matter.

Output side length; × denotes rejected shape or crop

valid / none	×	×	×	×	×	×	×	×	×	×	×
valid / symmetric	4	×	×	×	68	×	×	×	×	×	388
valid / asymmetric	4	4	4	4	68	68	68	68	68	68	388
same / none	×	×	×	×	×	×	×	×	256	×	×
same / symmetric	×	×	×	×	×	×	×	×	256	×	×
same / asymmetric	176	176	176	192	240	240	240	240	256	256	560
	188	189	190	200	252	253	254	255	256	257	572

Square input side length (pixels)

Figure 1: All 66 scalar cases. Entries are output side lengths in pixels; × is rejection, not missing data. Asymmetric means floor-left/remainder-right cropping. All 39 rejected cases remain visible. The 572 → 388 valid anchor is shape arithmetic only. Odd and incompatible even inputs are stress tests beyond the original even-prepool constraint.

Changing the crop policy changes executability, not Equation 3. Asymmetric same padding maps 252 and 253 to 240, and 256 and 257 to 256. Valid padding maps these four inputs to 68. Even the canonical valid-compatible input 572 becomes 560 in the same/asymmetric variant, and its same/no-crop variant fails at the deepest skip (71 versus 70). Conversely, 256 admits no-crop same padding but not symmetric-only valid cropping. The admissible input families have changed, rather than one family simply extending the other under identical conditions.

4.2 A compatible shape can still have a different boundary response

For all six numerical input sizes, valid convolutions return an all-ones output with minimum and maximum exactly 1. This is expected from the construction: averaging, max pooling, normalized up-convolution, cropping and normalized concatenated-channel mixing each preserve a constant one field when no zeros are inserted. This control provides a mechanistic reference without learned weights.

The same variant breaks that constant response at the image boundary. Figure 2 shows $n = 256$, where same padding needs no crop and output dimensions match the input. Nevertheless, the corner response is 0.11626864920076965, while the center and maximum response are 1. For 188, 189, 252 and 253, the same-padding minimum is 0.19086768475540394; for 256 and 257 it is 0.11626864920076965. Thus the same construction can produce a different boundary minimum across size/alignment regimes. These values reflect the joint specified operator, including pooling and crop alignment; they do not isolate a learned edge-detection mechanism or quantify an accuracy loss.

The comparison does not assert that the two entire output arrays should be subtracted pixelwise: their extents and alignment differ. It establishes that replacing valid convolutions changes the boundary response of the declared graph. In particular, the 256×256 same-padding case is a counterexample to the inference that successful concatenation and preserved input size imply preservation of the original boundary contract.

4.3 Structural support changes with location and alignment

Table 1 reports selected one-axis supports at input 256. For valid padding, the sampled outputs each have 188 contributing axis locations, but the intervals shift with output position. Same padding clips the corner support to 94 locations and gives the center a different interval. Even equal support counts do not establish equal spatial alignment: valid center index 34 has support 32...219, whereas same center index 128 has support 34...221.

Across all selected center-output impulse checks, 36 of 36 predictions agree at the fixed threshold, with zero mismatches. At input 188, a corner impulse reaches the valid center with response

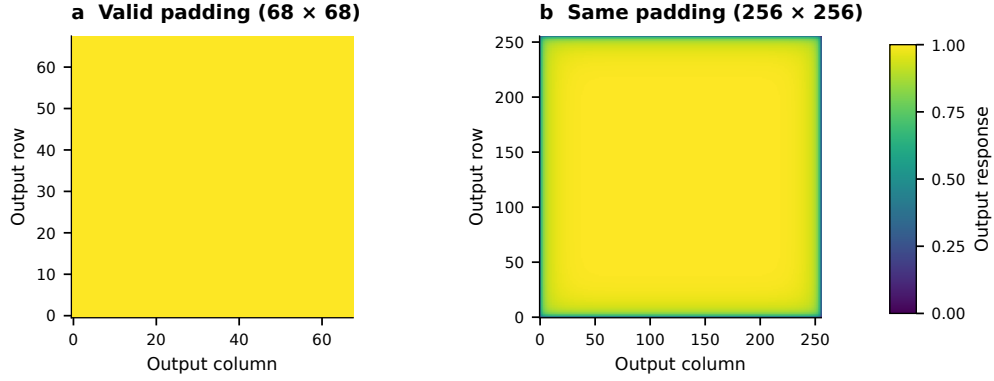


Figure 2: Measured first output channel for an all-ones 256×256 input in the reduced-channel fixed-weight network. The second channel is identical. The panels share a $[0, 1]$ response scale, but their axes refer to their own output indices: the equally sized panel boxes do not imply equal output extent. Valid output is uniformly 1 on 68×68 ; same output spans 256×256 with corner value 0.1162686492 and maximum 1. These are activation probes, not segmentation-quality measurements.

Table 1: Structural support at input side 256. Intervals are inclusive, zero-based input-axis indices; these particular sets contain all indices between their endpoints. The two-dimensional potential support is the Cartesian product. These are graph dependencies, not measured learned receptive fields.

Padding	Output location	Output index	Input-axis support	Count
Valid	First	0	0 ... 187	188
Valid	Center	34	32 ... 219	188
Valid	Last	67	64 ... 251	188
Same	First	0	0 ... 93	94
Same	Center	128	34 ... 221	188
Same	Last	255	162 ... 255	94

approximately 7.17×10^{-14} and the same center with approximately 2.09×10^{-12} ; both exceed the threshold. At input 256, corner and near-corner impulses do not reach either center, while center impulses do. These checks retain weak but nonzero dependencies rather than treating a visually blank output as zero.

Support is not described adequately by one global width. For example, the same-padding center at input 252 has axis support $18 \dots 221$, containing 204 positions, whereas the center at input 256 has 188. At input 257, the last same-padding output includes $162 \dots 256$ (95 positions), compared with 94 positions at input 256. Floor pooling, path union, and the chosen skip crop together determine these sets. The finite probes support the implementation’s selected predictions, while leaving untested every unprobed impulse and alternative padding/alignment policy.

5 Relation to U-Net and interpretation

The nearest reference is the original U-Net itself: the architectural description and spatial diagram define the geometry reproduced here, while its tiling discussion explains why context and output extent are separate choices [Ronneberger et al., 2015]. Our contribution is a bounded counterexample study of a substitution, not a new segmentation architecture. We preserve the original channel counts only in scalar traces and replace trained computation with a deliberately simple positive-weight probe. This distinction separates the present evidence from the original paper’s task-level evaluation.

The author distribution packages a trained Caffe network and associated tools [Ronneberger et al., n.d.]; those programs and weights were not executed here. Accordingly, reproducing a dimension

anchor is narrower than reproducing that implementation, its training procedure, or its published segmentation results. We also make no survey claim that all U-Net implementations choose these floor-pooling and crop conventions. Input padding, a different output-padding rule, resizing, or another boundary extension can define a useful but different contract.

The practical implication is to specify the intended contract before treating a padding change as a refactor. If output size must equal input size, Equation 3 shows exactly which tested construction needs an additional policy. If the intended prediction region requires real image context, zero extension changes what information is available at its edges. Crop offsets and boundary extension therefore deserve explicit documentation and tests alongside tensor sizes. Compatibility of a shape is a necessary interface condition for concatenation; it is insufficient evidence of operator equivalence.

6 Limitations and conclusion

This study uses eleven scalar sizes and six numerical sizes, all square, with one deterministic weight construction. It does not estimate a population effect and has no independent statistical replicates or confidence intervals. Its error count depends on the declared grid and policy choices. The input range does not characterize minimum feasible sizes or all possible failure classes. The 572 result is scalar only, and the original-width numerical network was not evaluated.

The numerical probe intentionally removes training and learned channel diversity. Positive averaging kernels make boundary effects observable but cannot establish how a trained model will compensate for them. Structural support describes possible paths and selected positive-impulse responses, not input-specific max-pool gradients or learned effective receptive fields. No accuracy, speed, peak memory, seamless-tiling experiment, or biomedical validity is measured. Comparisons of alternative boundary extensions and trained segmentation would require separate matched experiments.

Within these limits, replacing valid convolutions by zero-padded same convolutions does not preserve the tested U-Net spatial contract. Dimension recurrences and all retained scalar outcomes identify when pooling residues obstruct concatenation; numerical responses and selected structural checks expose a distinct boundary change after shape compatibility is achieved. An explicit crop policy can make the network executable, but that policy and the padding choice jointly define a new geometry and boundary behavior.

References

- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241, 2015. doi: 10.1007/978-3-319-24574-4_28. URL <https://arxiv.org/abs/1505.04597v1>. Architecture passages checked in arXiv:1505.04597v1.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation, project page. University of Freiburg, n.d. URL <https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>. Accessed 6 October 2026; original release not executed.

A Reproduction details and canonical trace

The supplied implementation is an independent NumPy construction. The scalar and numerical functions share their stated topology but use different channel schedules. The numerical study stores full output arrays as well as scalar summaries, allowing the response ranges and both identical channels to be inspected. The three declared crop policies are explored in scalar geometry; the numerical forward function always uses asymmetric cropping. Failures in the scalar sweep therefore must not be mistaken for failed executions silently removed from the numerical comparison.

For the canonical valid input 572, the encoder block side lengths are 568, 280, 136, 64, 28, with pooled sides 284, 140, 68, 32. Decoder up-convolution sides, from deepest to outermost, are 56, 104, 200, 392; the corresponding total skip crop differences are 8, 32, 80, 176. Decoder block outputs are 52, 100, 196, 388. These quantities follow Equations 1–2 and match the retained scalar trace. Their symmetric crop offsets are 4, 16, 40, 88 on each side.